Estructuras de Datos
Clase 1: Introducción

Dr. Sergio A. Gómez
http://cs.uns.edu.ar/~sag

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Trivia

- Asignatura: Estructuras de datos
- Profesor: Dr. Sergio Alejandro Gómez
- Email: sag@cs.uns.edu.ar
- CV:
  - Analista de Computación (Universidad Nacional de La Plata)
  - Licenciado en Informática (Universidad Nacional de La Plata)
  - Magister en Ciencias de la Computación (Universidad Nacional del Sur)
  - Doctor en Ciencias de la Computación (Universidad Nacional del Sur)
- Investigación:
  - Ontologías
  - Argumentación
  - Acceso a Datos Basado en Ontologías

Aprobación de la materia

- Cursada: Dos parciales + un recuperatorio final + un proyecto
- Promoción: Los que sacan A o B en los parciales, o C y A (en primero y segundo parcial en ese orden), rinden coloquio el día del recuperatorio
- Final: Los que no promocionan rinden examen final.
- Régimen de promoción: Ahora para rendir examen final los requerimientos de correlatividad son los mismos que para inscribirse.

Bibliografía

- M.T. Goodrich & R. Tamassia. Data Structures and Algorithms in Java. Fourth Edition. John Wiley & Sons, 2006.
- Nota: La lectura del libro es importante.
- Adicionalmente se usarán apuntes de la cátedra, videos y diapositivas en la práctica que se distribuirán oportunamente.
- Nota: Las diapositivas no son exhaustivas y no son para copiar y pegar código.

Bibliografía adicional

PDF que se lee mejor y versión más nueva del libro aunque algunos TDAs no coinciden en operaciones: M.T. Goodrich, R. Tamassia, Michael Goldwasser. Data Structures and Algorithms in Java. Sixth Edition. John Wiley & Sons, 2014.

Otro libro interesante: Mark Allen Weiss. Data structures and algorithm analysis in Java, Third Edition. Pearson, 2012

Libro viejo en castellano: Aho, Hoptcroft & Ullmann, Estructuras de datos y algoritmos (usaremos algunos algoritmos en pseudocódigo sobre el final de la materia).

En el CECOM dejamos la 2da edición de GT en <u>castellano</u>.

Motivaciones de la materia

- Estamos interesados en construir *buenas* estructuras de datos y *buenos* algoritmos.
- Estructura de datos = Manera sistemática de organizar y acceder datos.
- Me interesa insertar, modificar, eliminar, consultar y recorrer las estructuras de datos.
- Algoritmo = Procedimiento paso a paso para realizar una tarea en una cantidad finita de tiempo.

Objetivos

- Continuación y profundización de los conceptos vistos en IPOO.
- Aprendizaje de:
  - Estructuras de datos (ED) fundamentales
  - Elección de una estructura de datos para un dominio/problema determinado
  - Algoritmos asociados a las estructuras de datos
  - Evaluación de algoritmos (orden de tiempo de ejecución)

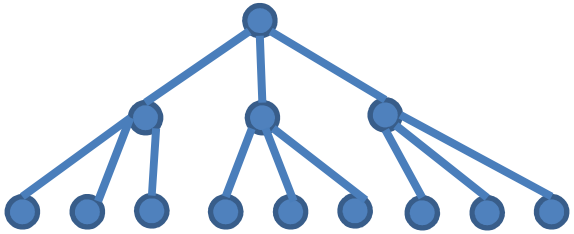
Repaso: Tipos de datos

- Un tipo de datos define un conjunto de valores y un conjunto de operaciones y viene equipado con una representación interna.
- Tipos simples: Sus variables mantienen un único valor (int, char, boolean, float, double)
- Tipos estructurados: Sus variables mantienen internamente más de un valor (string, arreglo, clase, archivo).
- En esta materia extenderemos estas nociones…

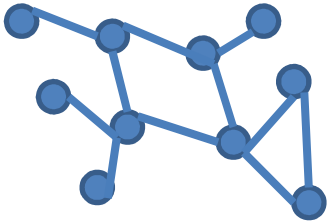
ED: Estructuras de datos conceptuales fundamentales

Secuencia (pila, cola, lista)

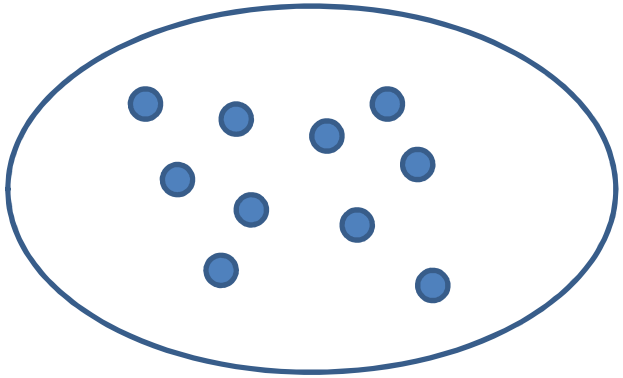
Jerarquías (árboles generales, árboles binarios)



Grafos/Redes



Conjuntistas: conjunto (Set), correspondencia (Map), diccionario (Dictionary) (variantes: hash, árbol binario de búsqueda, árbol 2-3, B, AVL)



Usaremos los conceptos de IPOO como clases, arreglos y matrices para implementar estas abstracciones.
La especificación de las operaciones de estas abstracciones se definirán como *tipos de datos abstractos* representados como *interfaces*.
Programaremos en un lenguaje orientado a objetos (Java)

Principios de diseño orientado a objetos

- Abstracción: Ocultar detalles irrelevantes para aumentar entendimiento y claridad (e.g. *noción de tipo de dato abstracto --- tipo definido en términos de operaciones*).
- Encapsulamiento/ocultamiento de información: Los componentes de software no deben revelar detalles internos de implementación (e.g. no declarar atributos de clase públicos cuando deben ser privados, prohibido castear para espiar estado interno de variables).
- Modularidad: Principio organizacional que dice que los componentes de software se deben separar en unidades funcionales (e.g. si necesito representar personas y fracciones, debo escribir 2 clases distintas y no una sola para las dos).

Discusión: Abstracción, ocultamiento y modularidad

```
public class Persona {
        public String nombre;   // Debe ser public, protected o private? Cuál es la diferencia?
        private float peso;
        public Persona(String n, float p) {
                nombre = n; peso = p;
        }
        public float getPeso() { return peso; }
        public void setPeso( float peso ) {
                this.peso = peso;   // Por qué necesito this acá?
        }
        public String toString() { return “Me llamo” + nombre + “ y ” + peso + “ kg.”; }
}
```

Atributos, campos, variables de instancia: nombre y peso

Modificadores de acceso: protected, private, public.

Palabra reservada this (permite referenciar el campo *peso* cuando hay una variable/parámetro con el mismo nombre que la eclipsa).

¿Qué identificadores (métodos y atributos) de la clase Persona puede referenciar un cliente?

¿Qué falta?

Factores de calidad del software

- Usaremos POO porque permite construir software de buena calidad.
- ¿Cómo evaluar la calidad del software?
  - Factores externos de calidad: Cualidad del software que puede ser detectada por un usuario (sin conocer el código fuente).
  - Factores internos de calidad: Cualidad del software que puede ser percibida por el programador (conociendo el código fuente).

Factores de calidad

- Correctitud: El programa respeta la especificación (para una entrada dada produce la salida esperada).
- Robustez: El programa es capaz de manejar entradas no contempladas por la especificación (se espera que se degrade “con gracia” o “gracefully”). Ej.: Recibe un string cuando espera un entero. Factor crítico en aplicaciones industriales.
- Extensibilidad: Capacidad de agregarle funcionalidad al software (ej: programar subclases para agregar operaciones)

Factores de calidad

- Portabilidad: Habilidad del soft de correr con mínimo cambio en otro hard o sistema operativo. (por ello usamos Java)
- Reusabilidad: Capacidad de usar el código en la construcción de otros programas (ej. Uso de sentencia import).
- Eficiencia: Capacidad del soft de requerir pocos recursos de cómputo (tiempo de CPU y memoria).
- Facilidad de uso: Capacidad del soft para ser usado por usuarios con distinta habilidad (ej. GUI).

Interfaces

- Una interfaz es una colección de declaraciones de métodos.

```
public interface nombre_interfaz
{
    public tipo método1( parámetros formales );
    ...
    public tipo métodon( parámetros formales );
}
```

```
// Vendible.java
public interface Vendible {
    public String descripcion();
    public int precioLista();
}
```

```
// Transportable.java
public interface Transportable {
    public int peso();
    public boolean peligroso();
}
```

Implementación de interfaces

- Una clase que implementa una interfaz debe implementar todos los métodos provistos por la interfaz.

```
public class MiClase implements NombreInterfaz
{
    …. Lista de atributos …
    public MiClase()  {  … código del constructor …  }
    public tipo método1( parámetros formales )
    { … código del método … }
     ...
    public tipo métodon( parámetros formales )
    { … código del método … }
}
```

Diseño con diagrama UML: Representación de clases e interfaces

Los programas representan el problema y el hardware al mismo tiempo.
Los diagramas UML representan los datos en alto nivel, en forma independiente del lenguaje de programación .

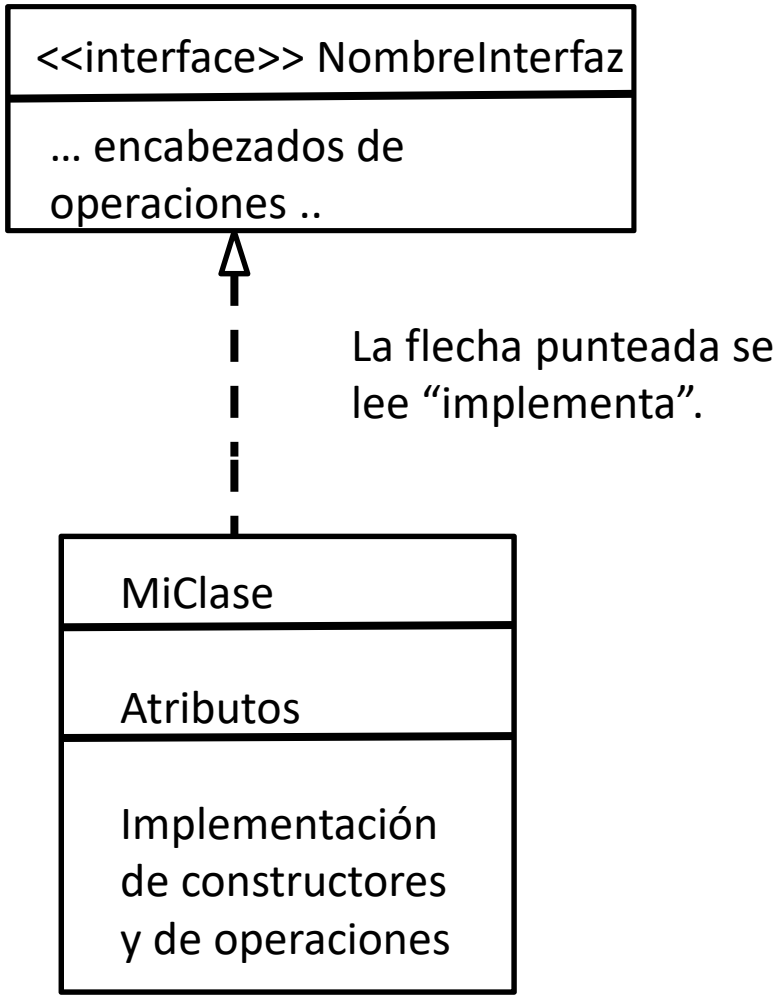
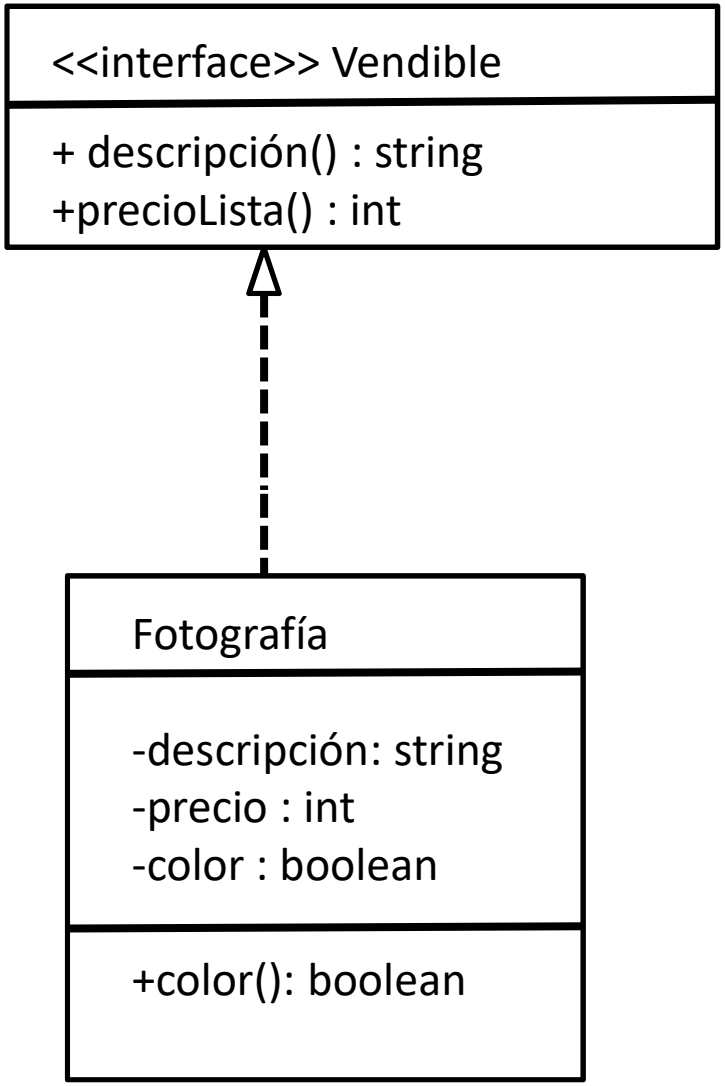


Diagrama UML

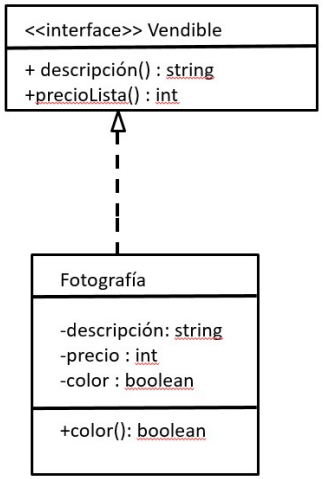
Notar la palabra *<<interface>>* para denotar una interfaz.

La flecha punteada se lee "implementa".

<u>Modificadores de acceso:</u>
+ significa *público*
- significa *privado*
significa *protected*



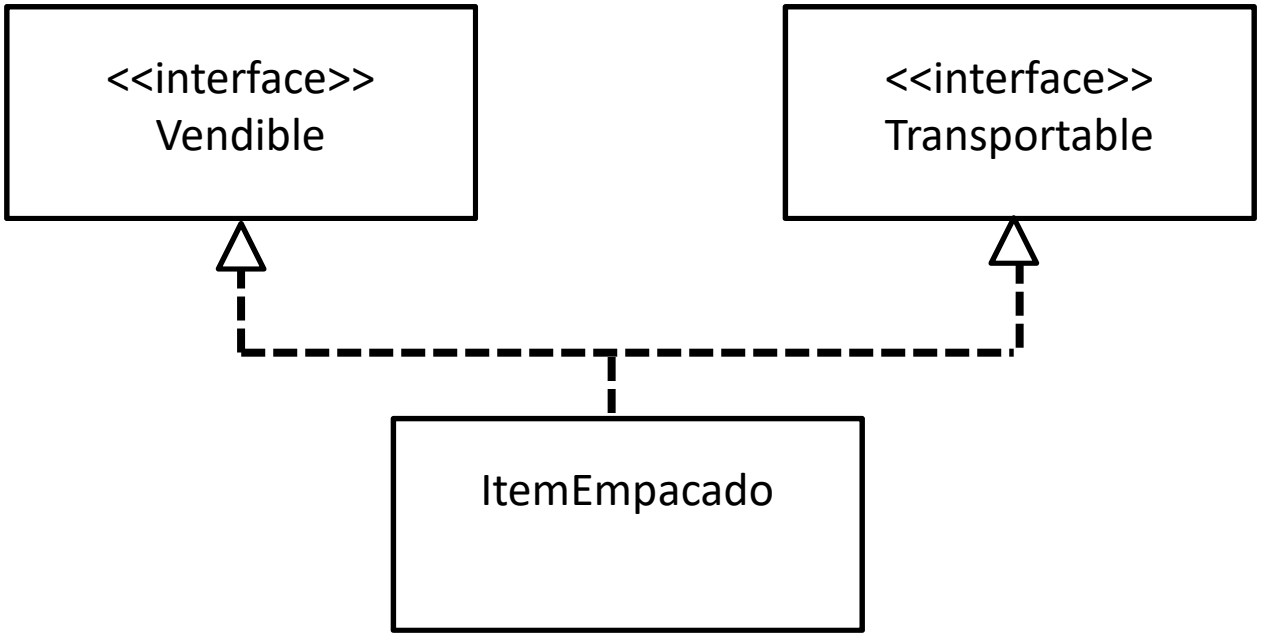
```java
// Fotografia.java
public class Fotografia implements Vendible {
    private String descripcion;
    private int precio;
    private boolean color;

    public Fotografia( String desc, int p, boolean c )
    { descripcion = desc; precio = p; color = c; }

    // Implemento los dos métodos de la interfaz vendible:
    public String descripcion() { return descripcion; }
    public int precioLista() { return precio; }

    // Agrego los métodos que considere necesarios:
    public boolean color() { return color; }
}
```



<u>Nota:</u> Flecha contínua significa *extends* mientras que flecha punteada significa *implements*.

Implementación de más de una interfaz

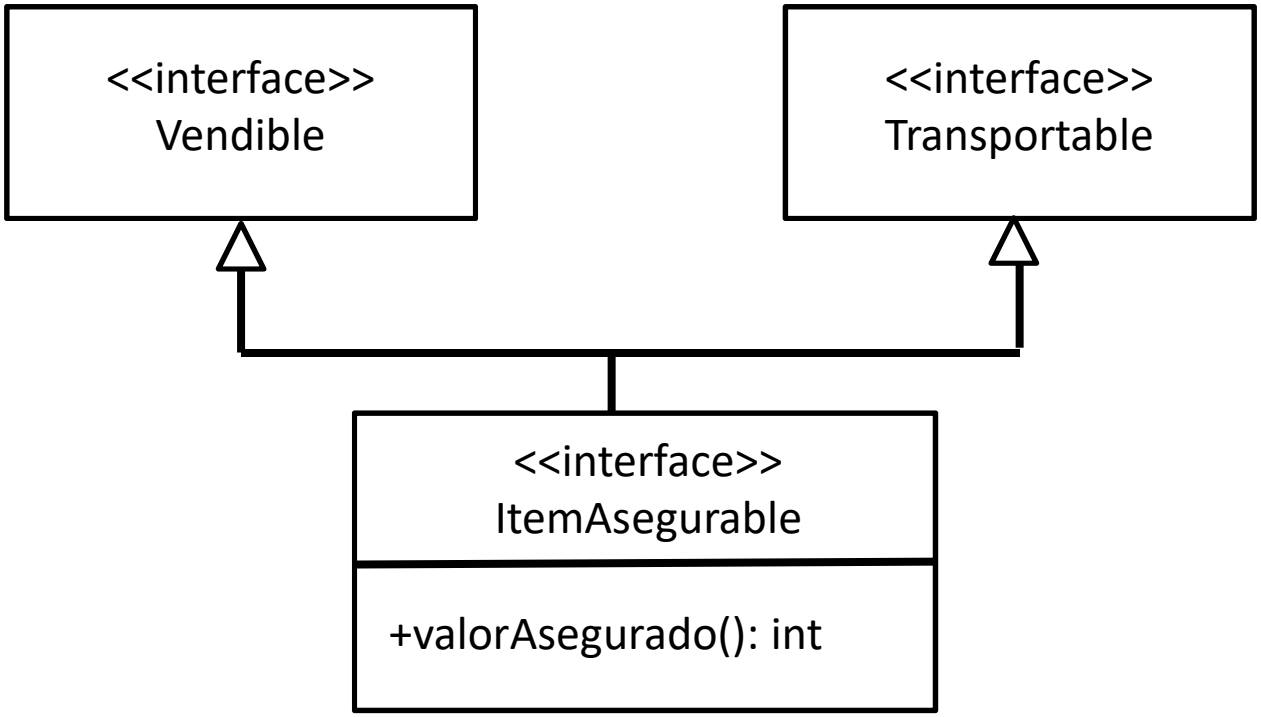
La clase *ItemEmpacado* implementa las interfaces *Vendible* y *Transportable*:




```
public class ItemEmpacado implements Vendible, Transportable {
    // Atributos:
    // Constructor:
    // Implementación de todos los métodos de ambas interfaces:
    // Implementación de métodos adicionales:
}
```

Herencia múltiple de interfaces

La interfaz *ItemAsegurable* extiende a las interfaces *Vendible* y *Transportable* y las extiende con el método *valorAsegurado*:




```
public interface ItemAsegurable extends Vendible, Transportable {
    public int valorAsegurado();
}
```

Ventajas del uso de interfaces

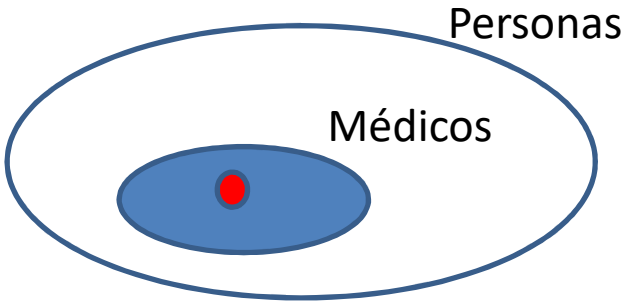
- Al usar una interfaz uno especifica "qué" debe hacer una clase pero no "cómo" debe hacerlo.
- Esto será útil para definir tipos abstractos de datos (defino operaciones pero no valores, recordar que en una clase debo definir atributos).
- Las interfaces no hacen suposiciones de "cómo" serán implementadas.
- Una vez que se define una interfaz, ésta puede ser implementada por cualquier número de clases.
- Implementar una interfaz requiere implementar todos los métodos definidos por la misma.
- Brindan una forma limitada y segura de herencia múltiple (una clase puede implementar más de una interfaz).

Interfaces versus clases abstractas

- Recuerde que una clase abstracta es una clase que tiene por lo menos un método abstracto (que no se implementa) y se señala con la keyword *abstract*.
- Una interfaz sería parecida a una clase abstracta donde todos sus métodos son abstractos.
- Las interfaces están pensadas para soportar la resolución de métodos en tiempo de ejecución igual que las clases abstractas (polimorfismo).
- El problema con las clases abstractas es que están pensadas para un modelo de herencia simple con lo que las mismas deberían estar más y más arriba en la jerarquía de clases oscureciéndola.
- Este problema se evita con las interfaces ya que dos clases no relacionadas pueden implementar la misma interfaz.
- Ejemplo: la clase Persona puede ser Transportable y la clase Reloj también.

Repaso de reglas de asignación en IPOO

Puedo asignar directamente variables de subclases en variables de superclases:



<u>Ejemplo:</u>

Si Medico es subclase de Persona:

Persona p;

Medico m = new Medico("House");

p = m; // Legal porque un médico es una persona

Medico m2 = p; // Ilegal porque a pesar de que p apunta a un objeto de tipo médico no se puede asegurar que una persona sea un médico.

<u>Solución:</u> Castear:

Medico m2 = (Medico) p;

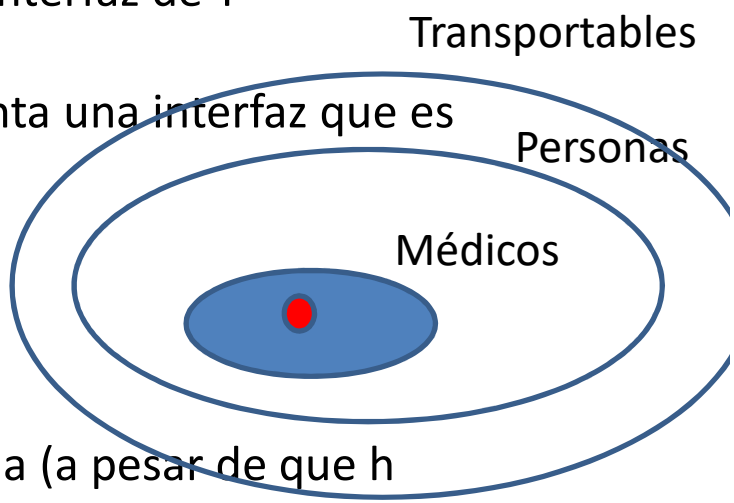
Si p apunta a un Medico o un objeto de alguna de sus subclases (Cirujano, Pediatra, …) funcionará, sino producirá un error (ClassCastException).

Estas reglas se extienden naturalmente a interfaces…

Reglas para asignar en forma válida: a = b;

Supongamos que *a* es de tipo *U* y *b* de tipo *T*. Es válido asignar cuando:

- "a" y "b" son del mismo tipo de clase o interfaz (i.e. *T* = *U*).
- Cuando *T* se convierte en un tipo U más amplio, se produce una conversión ***widening*** (ensanchante):
  - T y U son nombres de clases y U es superclase de T
  - T y U son nombres de interfaces y U es superinterfaz de T
  - T es una clase que implementa la interfaz U
  - (T es una subclase de una clase que implementa una interfaz que es subinterfaz de U).
- Son todas verificables por el compilador.



<u>Ejemplo:</u> Si la clase Medico es subclase de Persona:

```
Medico s = new Medico("Dr Strange");
Persona h = s;
```

Con h solo puedo usar los métodos de la clase Persona (a pesar de que h referencia a un Médico en ejecución).

<u>Ejemplo:</u> Si la clase Persona implementa la interfaz Transportable:

```
Transportable t = s;
```

Solo puedo ejecutar t.peligroso() y t.peso() porque t es Transportable (a pesar de que t referencia en ejecución a un médico).

Narrowing conversions (conversión angostante)

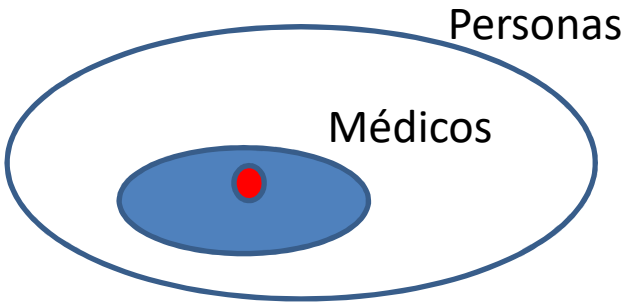
Ocurre cuando un tipo T se convierte en un tipo S más *angosto*.

Casos comunes:

- T y S son clases y S es subclase de T.
- T y S son interfaces y S es subinterfaz de T.
- T es una interfaz implementada por la clase S.

La asignación requiere un casting explícito y su correctitud se controla en ejecución:

```
T b = new T();
S a;
a = (S) b;
```



Ejemplo:

```
Persona p = new Medico("Dr House");
Transportable t = p;
Medico m = (Medico) t;      // Ok porque t apunta un médico.
```

Supongamos que la clase Perro implementa la interfaz Transportable:

```
t = new Perro("Fido");
m = (Medico) t;   // Falla en ejecución produciendo una ClassCastException.
```

Genericidad

Objetivo: Mejorar la reusabilidad del código.

Problema: Quiero programar una clase Colección una única vez y luego utilizarla para programar una colección de enteros, una colección de caracteres, una colección de personas, etc.

¿Cómo hago esto?

Primer requerimiento: Los valores de los tipos básicos no se almacenan como objetos y necesito tratar a todos los tipos de datos como objetos.

Segundo requerimiento: Quiero reusar la mayor cantidad de código posible.

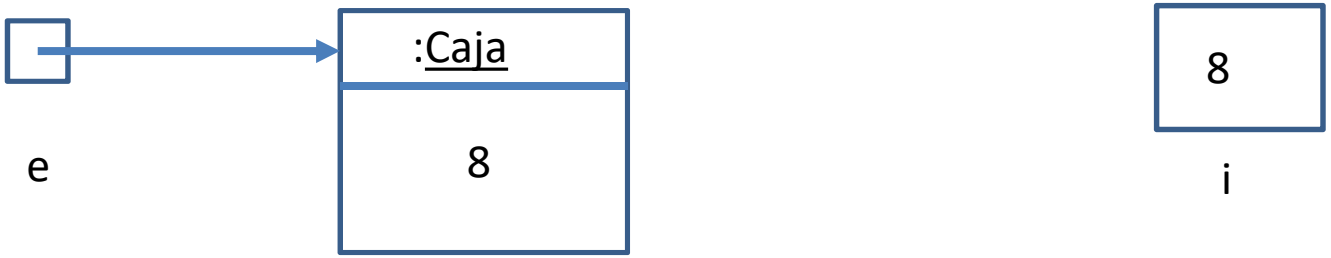
Manejar tipos básicos como objetos: Encapsular un entero en un objeto

```
public class Caja {
  private int valor;
  public Caja(int x) { valor = x; }
  public int getValor() { return valor; }
  public void setValor( int nuevo_valor ) { valor = nuevo_valor; }
}
```

Entonces para usar estos enteros habría que hacer algo así:

```
Caja e = new Caja(8);
int i = e.getValor();
```

Y en ejecución tendríamos algo así:



Nota: El problema de este enfoque es que necesitamos infinitas clases: una para representar una caja de enteros, otra para una de float, otra para una caja de Persona, etc.

Clases Wrapper

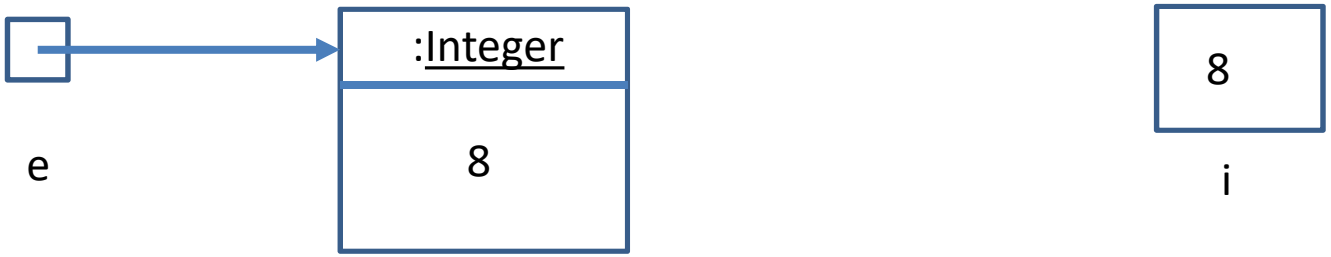
- En ocasiones es muy conveniente poder tratar los datos primitivos (int, boolean, float, char, etc.) como objetos.
- Las clases wrapper existentes son, Byte para byte, Short para short, Integer para int, Long para long, Boolean para boolean, Float para float, Double para double y Character para char.
- Observe que las clases envoltorio tienen siempre la primera letra en mayúsculas.
- <u>Nota:</u> Los objetos de clases wrapper son inmutables (i.e., no permiten actualizar los valores envueltos porque las clases wrapper no tienen setters).

Uso de clase wrapper Integer para envolver un int como objeto

La clase Integer de Java se comporta exactamente como la clase Caja anterior:

```
Integer e = new Integer(8);
int i = e.getValue();
```

Y en ejecución tendríamos algo así:

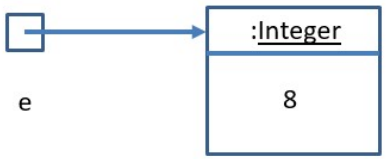


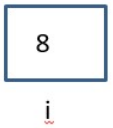
Esta forma de programar se ha desestimado (deprecated) y los compiladores lo indican prefiriendo hacer autoboxing y unboxing como mostramos en la diapositiva siguiente.

Autoboxing y Unboxing

Autoboxing: Conversión automática entre tipo primitivo y tipo wrapper (El valor de i se convierte en Integer)

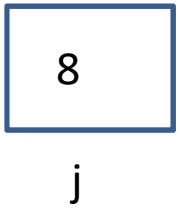
```
int i = 8;
Integer e = i;
```





Unboxing: Conversión automática entre tipo wrapper y tipo primitivo (el valor en e se extrae y almacena en j).

```
int j  = e;
```



Ojo: Los objetos wrapper son inmutables (su valor no cambia).

Entonces, las clases wrapper no sirven para simular pasaje de parámetros por referencia con tipos primitivos porque Java tiene pasaje de parámetros por valor (ver práctica 1).

Recordar: Los tipos primitivos tampoco se pueden pasar por referencia.

Genericidad con herencia (como en IPOO)

¿Cómo generalizo la idea de la caja para tener una caja de objetos de cualquier tipo?

```
public class CajaVieja {
        private Object dato;
        public CajaVieja (Object dato) {  this.dato = dato;  }
        public Object getDato() {   return dato;  }
}
```

En un programa cliente:

```
CajaVieja cajaEntero = new CajaVieja(8);    // Creo una caja con un 8 adentro
int x = (Integer) cajaEntero.getDato();   // Saco un object y lo casteo para poder usarlo
CajaVieja cajaString = new CajaVieja("Hola");   // Creo una caja con un string
String s = (String) cajaString.getDato();   // Saco un object y lo casteo para poder usarlo

int y = (Integer) cajaString.getDato();  // Compila pero produce error en tiempo de ejecución
                                         // de tipo ClassCastException porque me equivoqué
cajaEntero = cajaString; // Compila y se ejecuta aunque es incorrecto
// porque mezcla tipos de datos (el ambiente de ejecución no nota el error (es type unsafe)).
```

Otro problema resuelto con genericidad con herencia

- Problema: Implementar un par ordenado genérico.
- Usaremos un par de objects.
- Veremos que este enfoque no es type safe (no brinda seguridad de tipos).

```
public class ParOrdenado {
        protected Object x;
        protected Object y;

        public ParOrdenado(Object x, Object y ) {
                this.x = x;
                this.y = y;
        }
        public Object getFirst()  { return x; }
        public Object getSecond()  { return y; }
}
```

```
public static void main( String [] args ) {
        ParOrdenado p1 = new ParOrdenado(1,"Sergio");
        ParOrdenado p2 = new ParOrdenado(2.3f, 'a');

        // Para extraer datos del ParOrdenado hay que andar casteando:
        String s = (String) p1.getSecond();
        int i = (Integer) p1.getFirst();

        float f = (Float) p2.getFirst();
        char c = (Character) p2.getSecond();

        // Hasta acá todo bien a pesar de los casteos.
        // El siguiente error se detecta en ejecución y no en compilación:
        String s2 = (String) p1.getFirst();
        // Produce una ClassCastException.
}
```

Entonces, vamos a dejar de usar la *genericidad con herencia* y vamos a pasar a una solución mejor llamada *genericidad con polimorfismo paramétrico*.

Genericidad con polimorfismo paramétrico ☺

- Java 5.0 incluye un framework de genericidad para usar tipos de dato abstracto que evitan casting explícito.
- Un tipo genérico es un tipo que no es definido en compilación sino en tiempo de ejecución y permite definir una clase en función de parámetros formales de tipo que abstraen los tipos de algunos datos internos de una clase.
- Dada una clase genérica, vamos instanciar objetos usando parámetros actuales de tipo.

Genericidad con polimorfismo paramétrico

Ideas principales:

- Parametrizar definiciones de clases y métodos
- Brindar seguridad de tipos (el compilador realiza la verificación de tipos y previene errores de casteo en tiempo de ejecución).

Caja con genericidad paramétrica

Vimos que la CajaVieja de Object no funciona.

<u>Solución:</u> Parametrizar la clase con el tipo E de los datos almacenados.

```
public class Caja<E> {
    private E dato;
    public Caja(E dato) { this.dato = dato; }
    public E getDato() { return dato; }
    public void setDato( E nuevo_valor ) {  dato = nuevo_valor;  }
}
```

E se refiere a un tipo en particular.
El constructor toma un objeto de tipo E.
La función getDato devuelve un objeto de tipo E.

Para usar esta clase E debe ser reemplazado con una clase específica:

```
Caja<Persona> cp = new Caja<Persona>( p );    // cp es una caja de una persona
Caja<Float> cf = new Caja<Float>(4.5f);   // cf es una caja de un número real
Caja<String> cs = new Caja<String>("pepe");  // cs es una caja de un string
```

Uso de la caja parametrizada

```
Caja<Integer> cajaInt = new Caja<Integer>(8);
int x = cajaInt.getDato();  // No se necesita cast
Caja<String> cajaStr = new Caja<String>("hola");
String s = cajaStr.getDato(); // No se necesita cast
```

Las siguientes sentencias dejan de compilar:

```
String s = (String) cajaInt.getDato();
int y = (Integer) cajaStr.getDato();
cajaInt = cajaStr;
```

Los errores de ejecución ahora se capturan como errores de compilación.

Notas de genericidad con polimorfismo paramétrico

El nombre del parámetro de tipo del parámetro es indistinto.
Si quiero, puedo programar cambiando E por T y es lo mismo:

```
public class Caja<T> {
    private T dato;
    public Caja(T dato) { this.dato = dato; }
    public T getDato() { return dato; }
    public void setDato(T nuevo_valor ) {
                    dato = nuevo_valor;
    }
}
```

Notas de genericidad con polimorfismo paramétrico

Una clase puede tener múltiples parámetros:

class Cosas<A, B, C> { … }

Se puede hacer subclassing de clases parametrizadas:

class CajaEntero extends Caja<Integer> { … nuevos métodos ….}

Se puede extender un tipo parametrizado:

class CajaEspecial<E> extends Caja<E> { … nuevos métodos….}

La siguiente sentencia es legal:

Caja<String> cs = new CajaEspecial<String>("hola")

Porque caja especial es subclase de caja.

Notas de genericidad con polimorfismo paramétrico

Una clase parametrizada es como cualquier otra clase.

Puede ser usada como tipo de parámetros formales y como tipo de retorno de funciones:

```
Caja<String> método(int i, Caja<Integer> c) { … }
```

Con la caja, puedo simular pasaje por referencia con tipos inmutables:

```
void metodo2( Caja<Integer> c ) {
        // Modifico el estado interno de c con un setter:
        c.setDato( 8 );
}
```

Generidad con polimorfismo paramétrico

- Problema: Programar una clase para representar un par ordenado que permita almacenar dos valores de cualquier tipo.
- La clase será genérica, se llamará Pair y estará parametrizada en términos de los tipos de las dos componentes del par.
- (Solución presentada en el libro)

```java
/* Implementa un par ordenado (clave, valor) */
public class Pair<K,V>   // K y V son parámetros formales de tipo.
{
    // Los parámetros formales de tipo se pueden usar
    // para declarar atributos y variables:
    protected K key;   // key es tipo K
    protected V value;   // value es de tipo V

    // Los parámetros formales de tipo se pueden usar como
    // tipo en un parámetro formal:
    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    // Los parámetros formales de tipo se pueden usar como tipo
    // de retorno de una función
    public K getKey() { return key; }
    public V getValue() { return value; }
    public String toString() {
        return "[" +  getKey() +
        "," + getValue() + "]";
    }
}
```

Usando la clase Pair

- Crear un par de tipo (string, número real) y le almacenaremos el par (“Sergio”, 3.14):

```
Pair<String, Float> p = new Pair<String, Float>("Sergio", 3.14f);
```

- Crear un par de tipo (entero, carácter) y le almacenaremos el par (8, ‘b’):

```
Pair<Integer, Character> q = new Pair<Integer, Character>(8, 'b’);
```

- Crear un par de tipo (persona, persona) y le almacenaremos a Juan de 80 kg como primera componente y a Tito de 100kg años como segunda:

```
Pair<Persona, Persona> r = new Pair<Persona,Persona>(
        new Persona("Juan", 80.0f), new Persona("Tito", 100.0f) );
```

Nota: Usar tipos wrapper en la declaración y no tipos básicos. Ver que 3.14f de tipo float se convierte automáticamente en Float.

Conjunto con genericidad con polimorfismo paramétrico

- Problema: Implementar una colección genérica.
- Para ello utilizaremos como estructura de datos subyacente un arreglo de elementos de tipo E
- Luego crearemos un conjunto de enteros con el contenido { 1, 2 }.
- Luego crearemos otro conjunto de caracteres con los elementos { a, b }.

Usar una interfaz para especificar las operaciones en forma independiente de las posibles implementaciones.
De esta forma esta colección en un tipo de dato abstracto (solo especifico operaciones).

```
// Archivo: Coleccion.java (para completar con más operaciones):
// Note que la interface es genérica, tiene un parámetro formal de tipo E
// que indica el tipo de los elementos de la colección.

public interface Coleccion<E> {
        // Permite insertar item al final de la colección.
        public void insertar(E item);

        // Permite recuperar el elemento i-esimo de la colección
        public E recuperar(int i);
}
```

```
// Archivo fuente: ColeccionConArreglo.java
public class ColeccionConArreglo<E> implements Coleccion<E>
{
        protected int cant;    // cantidad de elementos almacenados
        protected E [] datos;  // arreglo de elementos

        public ColeccionConArreglo(int tamanioMaximo)       {
                cant = 0; // Hay 0 elementos al crear la colección
                datos =  (E []) new Object[tamanioMaximo]; // produce warning
        }

        public void insertar(E item) {
                if( cant == datos.length ) {
                        // Incrementar tamaño del arreglo datos
                        // Tarea para el lector
                }
                datos[cant++] = item;
        }
```

// Archivo fuente: ColeccionConArreglo.java continuación:

```
        // Precondición: i está entre 0 y la cantidad de elementos-1
        public E recuperar(int i) {
                return datos[i];
        }
}
```

Discusión: ¿Cómo hace el cliente para ver cuántos elementos hay en la colección? ¿Qué operación falta programar para esto?
Ver en la práctica que se piden más operaciones.

Detalles para resolver en próximas clases:
¿Qué ocurre si el cliente no envía un “i” en rango adecuado => Excepciones para manejar errores y tener código robusto

```
// Archivo fuente: Test.java (nueva versión)
public class Test
{
  public static void main( String [] args ) {
        // Creo una colección de enteros.
        // Debo usar la clase wrapper Integer porque los parámetros
        // actuales de tipo no pueden ser tipos básicos.
        // Note como asignamos un objeto de tipo clase
        // ColeccionConArreglo  a una variable de tipo interfaz Colección:
        Coleccion<Integer> conj_int =
            new ColeccionConArreglo<Integer>(5);

        conj_int.insertar( 1 ); // autoboxing de int a Integer
        conj_int.insertar( new Integer(2) ); // Alternativa redundante

        int i = conj_int.recuperar( 1 ); // Sin casting!
        System.out.println( "x: " + i );  // Imprime x: 2
```

```
// Archivo fuente: Test.java … continuación

        // Creo una colección de caracteres.
        Coleccion<Character> conj_ch =
            new ColeccionConArreglo<Character>(5);
        conj_ch.insertar( 'a' );  // Autoboxing de char a Character
        conj_ch.insertar( 'b' );
        char c = conj_ch.recuperar( 1 );

        // Sin casting! + unboxing de Character a char (i.e. recuperar
        // retorna un Character que es convertido a char
        // para hacer la asignación).
        System.out.println( "c: " + c ); // Imprime: c : b
    } // Fin de main
} // Fin de clase Test.
```

Sumario

- Contexto curricular: 3era materia de programación.
- Objetivos: Programar ED + algoritmos y evaluarlos.
- Factores de calidad: garantizan buenas soluciones, OOP (correctitud, legibilidad, mantenibilidad, portabilidad,, reusabilidad, robustez, usabilidad, EFICIENCIA)
- Interfaces: Generaliza la noción de clase abstracta (interface, implements, extends, herencia múltiple de interfaces).
- Clases Wrapper: Usar tipos básicos como objetos (Character, Integer, Float, Double, Boolean, autoboxing, unboxing).
- Genericidad con polimorfismo paramétrico: Mejorar reusabilidad (class Pair<A,B>, class Coleccion<E>, instanciar con Pair<Integer, Float>, Coleccion<Persona>)

- Clase basada en Capítulos 1 y 2 de Goodrich y Tamassia (leer luego de entender diapositivas)